

CASE STUDY • MINING SOFTWARE REPOSITORIES

Decoding the Issue Resolution Process in Practice

via Issue Report Analysis — A Case Study of Mozilla Firefox

Original paper by

Antu Saha • Oscar Chaparro

William & Mary

Presented by

Lucas Neves Lima

Why study issue resolution?

Issue trackers (Bugzilla, Jira, GitHub Issues) sit at the center of software maintenance. The literature describes issue resolution as a clean sequence — reproduce, analyze, design, implement, verify — but how developers actually carry it out in practice has remained unclear.

01

Theoretical View

A linear, sequential workflow assumed by textbooks and process models such as Rajlich's incremental change.

02

The Gap

No empirical evidence on how the process is actually executed and discussed by developers in practice.

03

This Study

Reveal the real process through qualitative and quantitative analysis of Firefox issue reports.

Research questions

The study analyzes 356 Mozilla Firefox issue reports to answer five research questions:

RQ1

What issue resolution stages are found in issue reports?

RQ2

How do the resolution stages interact with each other?

RQ3

What is the overall process of issue resolution?

RQ4

What resolution patterns are found in issue reports?

RQ5

What are the potential use cases of the derived patterns?

Data collection & qualitative coding

Subject system

Mozilla Firefox — a mature, widely-used open-source project with 19+ years of evolution and well-documented practices. Data drawn from BMO (Bugzilla) for the Core and Firefox components, covering January 2010 to April 2023.

Coding procedure

Iterative, multi-coder open coding by 7 annotators (6 PhD students + 1 professor). Each report was annotated and validated by at least two coders. Disagreements were resolved by consensus, with a third reviewer when needed.

Dataset at a glance

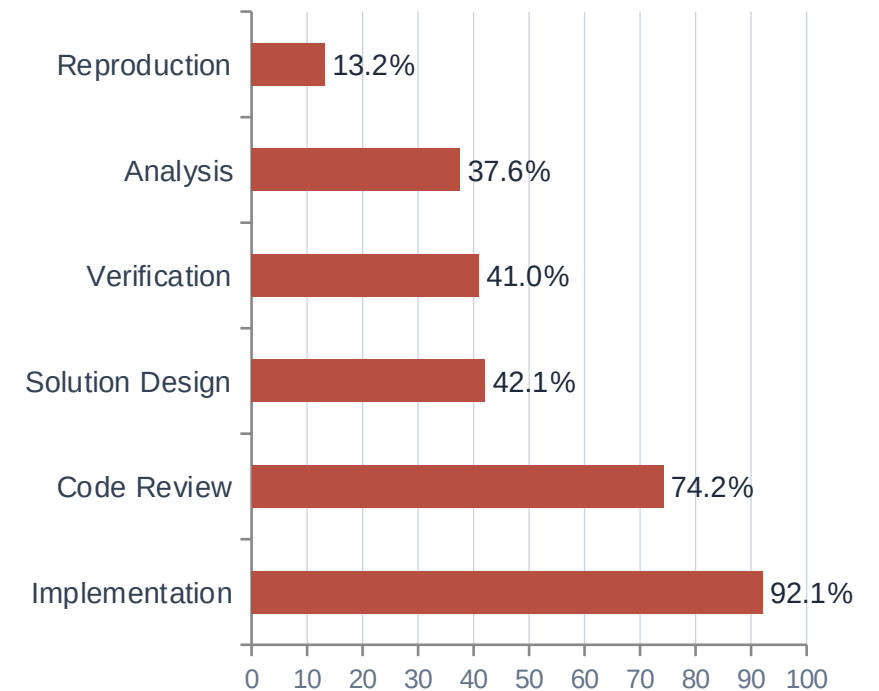
Total downloaded issues	199,271
Issues sampled & analyzed	356
Confidence level / error margin	95% / 5%
Annotated text snippets	3,707
Annotated comments	2,574
Inter-coder agreement	93%
Cohen's kappa	0.92
Codes derived	22
Problem categories	17

Six issue resolution stages identified

Open coding identified six distinct stages. They appear in reports with widely varying frequency.

Stage	Description	Issues
Reproduction (R)	Attempts to reproduce the reported problem.	13.2%
Analysis (A)	Reviewing the problem, locating code, identifying causes.	37.6%
Solution Design (SD)	Proposing or reviewing a potential solution.	42.1%
Implementation (I)	Making the code changes that resolve the issue.	92.1%
Code Review (CR)	Inspecting the implemented changes.	74.2%
Verification (V)	Testing the changes to confirm the issue is fixed.	41.0%

Frequency across 356 reports (%)



How stages connect: the most frequent transitions

Of 1,430 bigrams identified in stage sequences, nine transitions account for 80.6% of all stage switches.

Transition	Occurrences
I → CR (Implementation → Code Review)	403
CR → I (Code Review → Implementation)	187
SD → I (Solution Design → Implementation)	133
CR → V (Code Review → Verification)	94
I → V (Implementation → Verification)	86
V → I (Verification → Implementation)	73
A → I (Analysis → Implementation)	70
A → SD (Analysis → Solution Design)	65
CR → SD (Code Review → Solution Design)	41

Three recurring scenarios

Iterative QA loop

Implementation ↔ Code Review ↔ Verification form a tight cycle of feedback and revision.

Reproduction & Analysis pair

Although uncommon overall, when both R and A appear, they co-occur in 73.5% of those reports.

Analysis feeds design

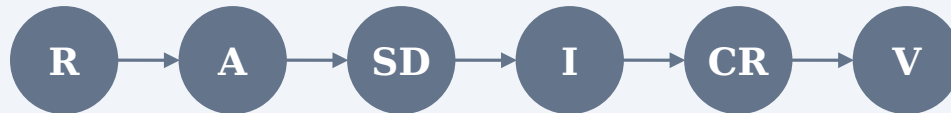
A → SD and A → I are common starting points, channeling investigation into action.

The real process is iterative, not linear

Aggregating the bigrams into a transition graph reveals a process that loops between stages rather than flowing top-to-bottom. Developers move back and forth as the problem demands.

Theoretical process

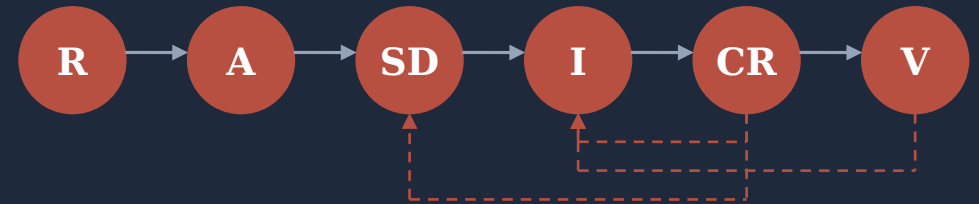
Linear, sequential (literature & documentation)



One direction only. Each stage handed off cleanly to the next.

Observed process at Firefox

Iterative graph derived from 356 reports



Dashed lines = iterative back-and-forth between stages. Implementation, Code Review and Verification form the central loop.

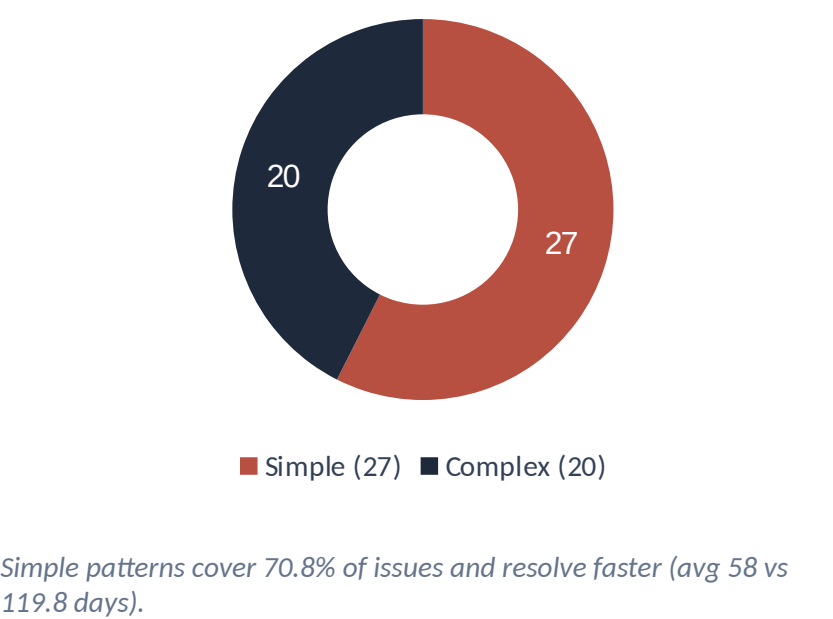
47 distinct resolution patterns

Qualitative grouping of stage sequences yields 47 patterns — concrete instances of the iterative process. 18 of them (the most recurrent) cover 80.6% of all 356 reports.

Top 8 most frequent patterns

Pattern	Complexity	Issues
I, CR, I?	Simple	64
A, I, (I CR V)?	Simple	32
(I, (CR V))+	Complex	28
SD, I, CR, (I V)?	Simple	24
A, SD, I, (I CR V)+	Simple	22
I	Simple	21
I, CR, V, I?	Simple	16
SD, (I, (CR V))+	Complex	13

Pattern complexity

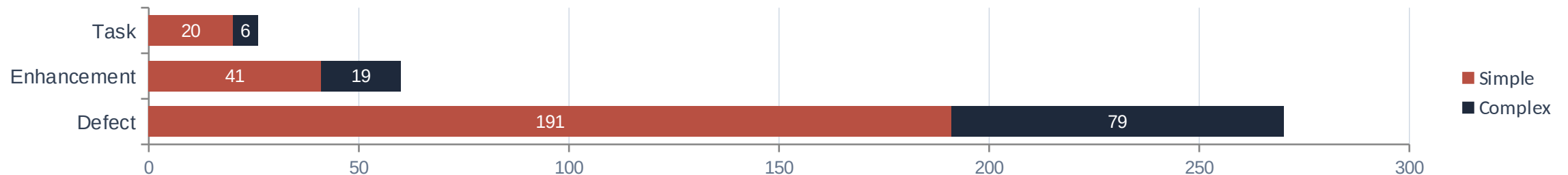


Simple vs. complex patterns: measurable effort gap

When sequences become longer and more iterative, resolution effort rises sharply. Differences are statistically significant (Mann–Whitney U, $p \approx 0.0$).

Metric	Simple patterns	Complex patterns
Issues covered	252 (70.8%)	104 (29.2%)
Resolution time (avg / median, days)	58 / 5	119.8 / 19.5
Stages in process (avg / median)	2.9 / 3	9.9 / 9
Commentators per issue (avg / median)	4.4 / 4	7.4 / 7

Pattern use by issue type



Six practical use cases identified by Mozilla developers

Two Mozilla engineers (7–11 years of experience) reviewed the patterns and identified six concrete applications:

01

Early detection of complexity

Spot issues whose resolution snowballs through repeated cycles before they consume excessive engineering time.

02

Process anomaly flagging

Highlight issues missing expected stages — for example, bugs needing third-party human verification.

03

Identifying problem components

Track resolution complexity per module to surface technical debt and refactoring candidates.

04

Strengthening triage bots

Improve heuristics in tools like Mozilla's bugbug to detect issues likely to be unsolvable or to stall.

05

Decomposing meta-issues

Recognize large, complex issues that should be broken into smaller, independently solvable sub-issues.

06

Onboarding & training

Show junior developers that resolution is iterative, not linear — multiple review and verification cycles are normal.

Generalizability & threats to validity

Generalizability beyond Firefox

A complementary case study annotated 20 issue reports from Chromium and GnuCash. All 7 patterns found in Chromium align with Firefox patterns; 9 of 10 GnuCash patterns also align. This suggests partial generalization across open-source projects of different scales and governance.

Cross-project validation

7 / 7

Chromium patterns aligned with Firefox

9 / 10

GnuCash patterns aligned with Firefox

Threats to validity

Incomplete records

Issue reports may not capture every activity — some are discussed elsewhere or omitted.

Researcher subjectivity

Mitigated by a multi-coder protocol with 93% agreement and Cohen's $\kappa = 0.92$.

Limited scope

Firefox is one project; full generalization requires larger-scale studies across systems.

CONCLUSION

Issue resolution is richer
and more iterative than theory suggests.

6

stages identified

from open coding of 356 reports

47

resolution patterns

18 cover 80.6% of all issues

70.8%

simple patterns

resolve issues in 58 days on average

29.2%

complex patterns

double the time, triple the stages

Implications: better tooling for traceability, anomaly detection, training of newcomers, and rationale management.

Thank you.

Lucas Neves Lima · Based on Saha & Chaparro, ICSE 2025